

GPIO

GPIO stands for General Purpose Input/Output. In the PIC16F72, the GPIO pins are mainly available through PORTA, PORTB and PORTC. PORTA has 6 pins, while PORTB and PORTC have 8 pins each, giving a total of 22 I/O pins.

The main purpose of GPIO is to allow the microcontroller to communicate with external hardware. A pin can be configured as an input when we want the microcontroller to receive a signal, or as an output when we want it to control something.

The direction of each pin is controlled using the TRIS registers. There are three registers called TRISA, TRISB and TRISC. If a TRIS bit is set to 1, the corresponding pin becomes an input. If it is set to 0, the pin becomes an output.

The PORT registers are used to read the input pins or control the output pins. For example, if a switch is connected to RB0, the microcontroller can read the condition of the switch through PORTB. Similarly, if an LED is connected to RB1, the microcontroller can make RB1 HIGH or LOW to control the LED.

Some GPIO pins also have other functions. For example, PORTB has interrupt-related features and PORTC pins can be used by different peripherals such as timers, PWM and serial communication. So, depending on the configuration, the same physical pin can perform different functions.

Timer

The timer is used when the microcontroller needs to measure time, create delays or perform an operation after a particular time interval. The PIC16F72 has Timer0 and Timer1, which can be used for different timing requirements.

Timer0 is an 8-bit timer/counter. It can work using the internal instruction clock or an external signal. When the timer is enabled, its value increases with the selected clock source. Since it is an 8-bit timer, it counts from 0 to 255 and then overflows back to 0.

Timer0 is useful for creating delays and for counting external events. An important point is that its overflow can also be used to generate an interrupt.

Timer1 is a 16-bit timer, so it can count over a much larger range compared to Timer0. It is useful when a longer time period or more accurate timing is required. Timer1 can also work as a timer or counter depending on the configuration.

The timers are connected to the internal clock and other parts of the microcontroller. The CPU does not always have to manually count every clock cycle. Instead, the timer can keep counting in the background and inform the CPU when a particular event occurs.

Interrupt Mechanism

An interrupt is a way for the microcontroller to temporarily stop its normal program execution and respond to an important event. Instead of continuously checking whether something has happened, the interrupt mechanism allows the hardware to notify the CPU when an event occurs.

For example, suppose an external switch is connected to the interrupt pin. When the switch causes the required signal change, an interrupt can be generated. The CPU temporarily stops its current program, goes to the interrupt service routine and performs the required operation. After finishing the interrupt routine, the CPU returns to the point where it stopped and continues the main program.

The PIC16F72 has several possible interrupt sources. These include external interrupts, timer overflow interrupts and interrupts related to some of the internal peripherals. RB0 can be used as an external interrupt pin, while Timer0 and Timer1 can generate interrupts when they overflow.

The interrupt mechanism is controlled using special registers that contain interrupt enable and interrupt flag bits. The enable bit decides whether a particular interrupt is allowed, while the flag bit indicates that the event has occurred.

The basic working can be understood as follows. First, an interrupt source is enabled. When the required event happens, its interrupt flag is set. The CPU detects the interrupt and jumps to the interrupt service routine. After handling the event, the interrupt flag is cleared and the CPU returns to the main program.